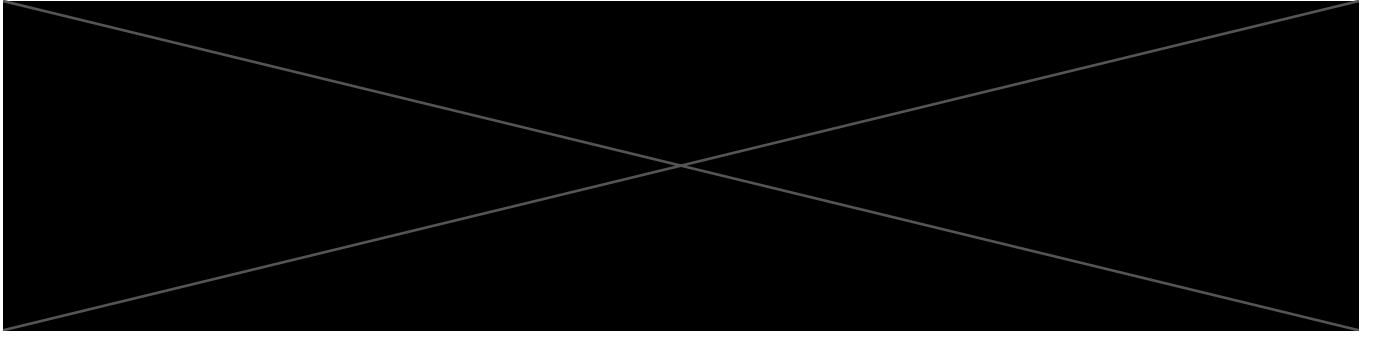




1 Methodology

To accurately evaluate the performance of our parallel program using George's data structure, we tested our algorithm in the following manner. First, keeping a fixed number of processors $p = 16$ and a fixed sparsity parameter $s = 0.01$, we compared the run-time of the program with varying matrix sizes, specifically with sizes $n = 2000, 4000$, and 8000 . Next, we fixed the matrix size to $n = 10000$ while changing the number of processors $p = 2, 4, 8, 16$ and the sparsity parameter $s = 0.1, 0.01, 0.001$.

2 Algorithm Overview

To generate the proper matrices following George's data structure, we created a custom *toop* struct that can store the row, column, and value of each nonzero element in the matrix. Then, we were able to create the matrices A and B by having them be vectors of the *toop* data structure. Since we have a custom struct, it was necessary to also create a custom MPI datatype to perform all-to-all on matrix B . We named this new datatype *toopType*. After some doing some overhead and performing the collective all-to-all communication, we embedded our communications into a ring topology by creating a new, 1-dimensional periodic Cartesian communicator. This way, we can easily shift the rows and columns of the matrices to make them match and perform dot products. As for the matrix multiplication, in our case, we used a two-pointer approach in which the rows of A perform dot products with the appropriate columns of B and correctly adding the result in the proper position in matrix C . Similar to how the data was distributed in matrices A and B , the resulting matrix C is distributed such that every processor holds $\frac{n}{p}$ rows of C . For more in-depth information regarding the overhead for each important step in the program, please refer to the provided README file.

3 Runtime and Space Analysis

We will first provide a theoretical analysis of our algorithm.

In terms of Space Complexity, each processor stores $\frac{sn^2}{p}$ elements of A and B and holds $\frac{n^2}{p}$ elements of C. Each instance of our toop struct is $32+32+64 = 128$ bits. We also store several smaller vectors of size n and p but this doesn't contribute in a significant way to our space complexity. In total our space complexity would just be $O(\frac{n^2}{p})$.

First we perform a Transpose of our B Matrix which will just be the runtime of All-to-all which is $O(\tau \log p + \mu mp \log p)$ with a message size of $\frac{sn^2}{p}$, which amounts to $O(\tau \log p + \mu sn^2 \log p)$ assuming hypercubic permutations.

Next we do an intermediary All-Reduce with a message size of 1 and runtime $O(\tau \log p + \mu \log p)$ and All-Gather with a message size of 1 and runtime $O(\tau \log p + \mu p)$. Meaning the All to all communication will dominate in terms of time complexity.

For the actual matrix multiplication, we do p rounds of smaller matrix multiplications of $\frac{n}{p}$ rows of A and $\frac{n}{p}$ columns of B. These multiplications will take $O(\frac{n}{p}(\frac{n}{p}(sn + sn)))$. Since each two pointer vector dot product takes $O(sn + sn)$ and we have $\frac{n}{p}$ rounds for each column of B and $\frac{n}{p}$ rounds for each row of A. Putting this together we get a runtime of $O(\frac{sn^3}{p})$ computation time. The communication time for each send and receive would just be $O(\tau p + \mu sn^2)$ which is still dominated by the All-to-All communication.

This means we have the following runtime for our algorithm.

$$\text{Communcation Time : } O(\tau \log p + \mu sn^2 \log p)$$

$$\text{Computation time : } O(\frac{sn^3}{p})$$

We can compare this to the serial matrix multiplication algorithm which takes $O(n^3)$ time using efficiency.

$$\text{Efficiency : } \frac{n^3}{p * (\frac{sn^3}{p} + \tau \log p + \mu sn^2 \log p)} = \Theta(1)$$

Implies to achieve optimal efficiency we must use.

$$p \log p = O(n)$$

$$p = O(\frac{n}{\log n})$$

This means that we can use up to $\frac{n}{\log n}$ processors and still achieve optimal efficiency.

4 Experimental Results

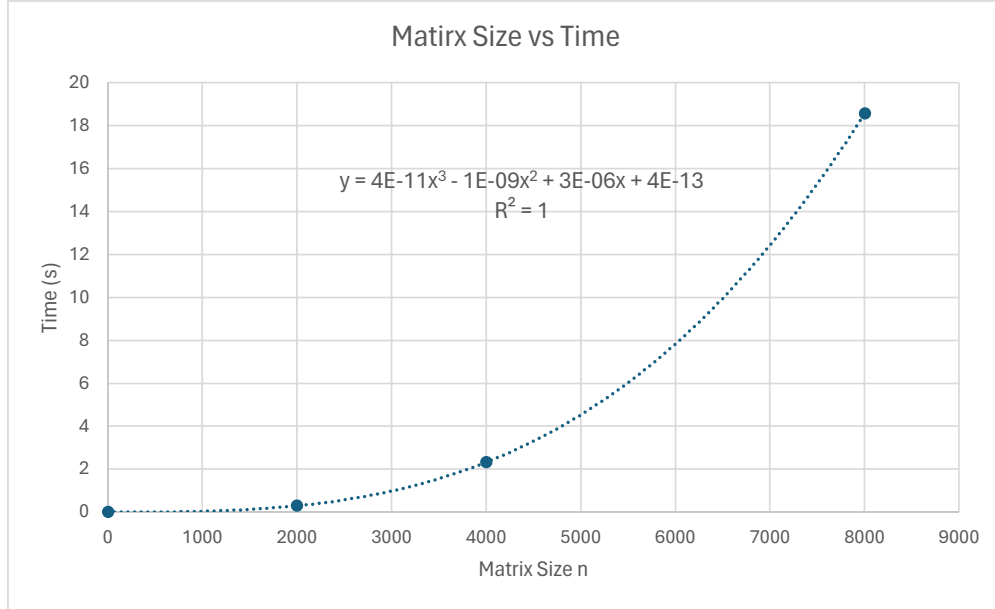


Figure 1: The dominating term is x^3 showing a $O(n^3)$ relationship between the matrix size n and runtime. This is consistent with our theoretical runtime analysis.

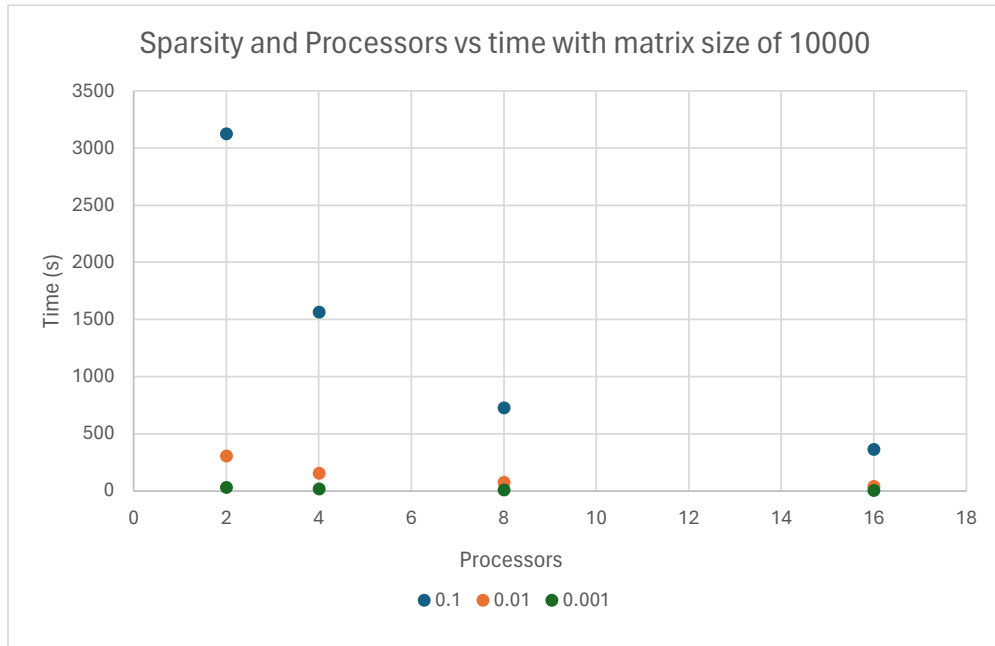


Figure 2: Graph shows performance for various values of Sparsity and Processors with a fixed matrix size. We see an increased performance with more processors and a more sparse matrix.

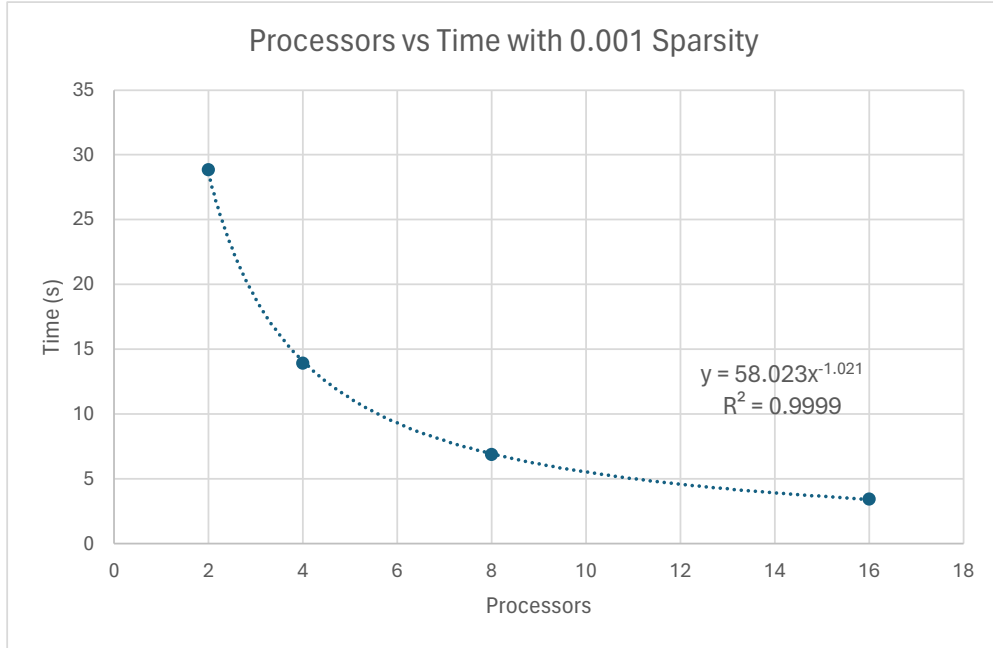


Figure 3: Here we examine that the relationship between p and time is $O(p^{-1}) = O(\frac{1}{p})$ with an R^2 value of .999. This is consistent with our theoretical runtime analysis.

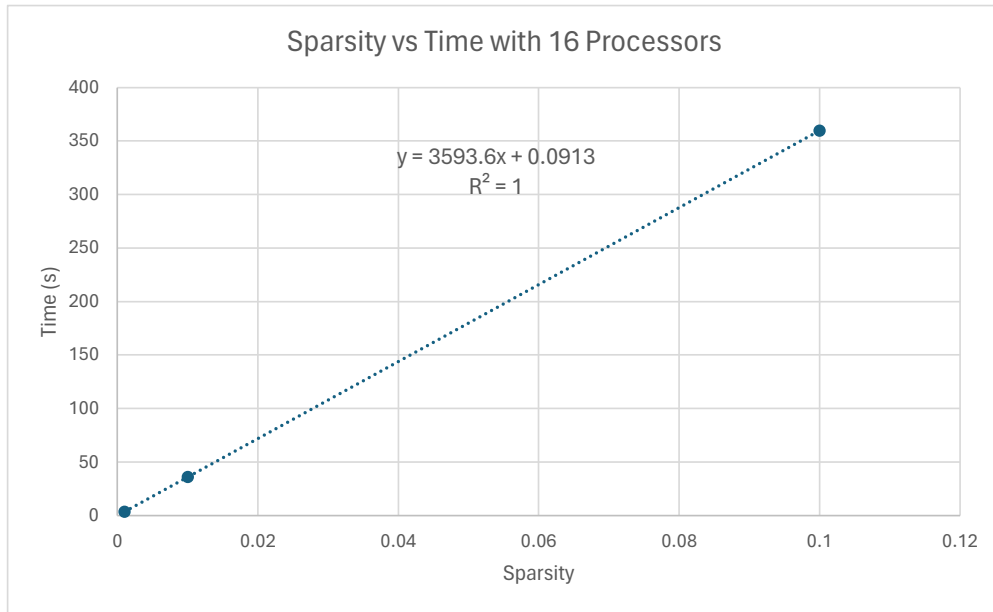


Figure 4: Here we see a linear relationship between sparsity and time suggesting a $O(s)$ relationship which is consistent with our theoretical runtime analysis.

5 Observations

From our experimental results, we observe that doubling the matrix size n increases the run-time of the algorithm for a fixed p and s . The increase in time as we go from $n = 2000$ to 4000 and to 8000 follows the trend of traditional matrix multiplication algorithm of $O(n^3)$. With $n = 2000$ finishing in 0.2927 seconds, $n = 4000$ of 2.3203 seconds, and $n = 8000$ of 18.5602 seconds. As seen in the Figure 1, we are able to fit a polynomial with a leading degree of 3 with an R^2 value of 1. This is consistent with our theoretical runtime analysis.

In Figure 3, we show how with a fixed sparsity value and fixed Matrix value of 10000, an increase the number of processors results in a $O(\frac{1}{p})$ relationship with runtime. We fix a p^{-1} trend line to our data with an R^2 value of .9999 suggesting a very strong fit. This means that our algorithm will scale nicely as we increase the number of processors meaning we can take full advantage of large clusters to perform multiplication of very large sparse matrices.

In Figure 4, we show the linear relationship with sparsity and performance with fixed matrix size and processor count. This shows that our algorithm performs much better in sparse matrices than dense ones. This is consistent to our theoretical runtime analysis.

In Figure 2, we get to see all of these variables at play in a single graph. Notice that when s is just .1, using even 4 processors runs in over 1500 seconds for a matrix size of 10000. However, if we have a very sparse matrix with an $s = .001$, it only takes 14 seconds. We can scale the number of processors to improve our runtime regardless of sparsity meaning this algorithm is efficient, however it is not practical for small number of processors and large dense matrices.

